research · 2026-09-17

a model that answers in probabilities

Adrian Tame Jacobo, writing for Plicara Labs · 2026-09-17 ·

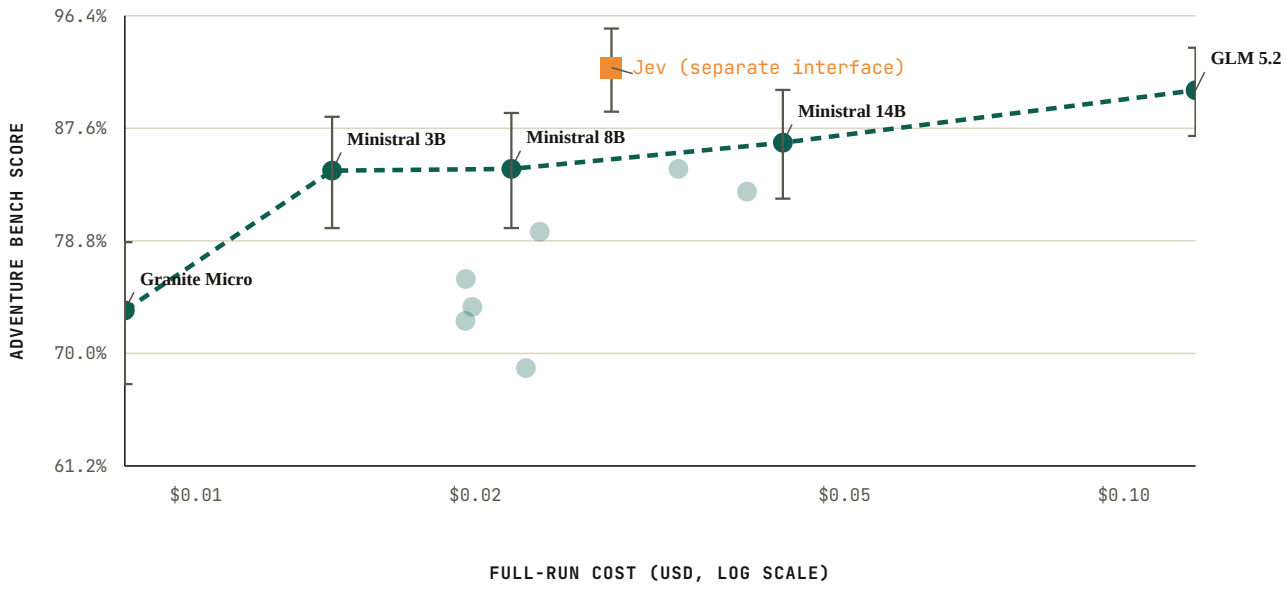
Everything awkward about putting LLMs inside programs lives in the gap of their natural output: language. Traditional software is not designed to take natural language as input, and that is why LLMs as part of pipelines can break things. Examples include but are not limited to coaxing text into JSON, parsing it back, and generating valid URLs. TypeSafe's Jev is new and pretty exciting, and kind of breaks this calculus. It never generates text at all. You send it application state and typed questions, and it returns choices, scores and probabilities your code can branch on directly.

One of the use cases they included in their demo of the model was it playing Doom at a ridiculous speed, and I run a benchmark that could really benefit from something like that. So we ran Jev on AdventureBench, our test of grounded instruction-following for text-based adventure games: map a player's free-form command onto a small action vocabulary against a described scene, or refuse honestly when nothing fits.

results: incredibly accurate

There is one methodological point to get out of the way. Our benchmark's comparison contract requires all models in one release to share the same prompt. Jev cannot take our chat prompt since it has no chat interface. Scoring it through the chat harness would mean testing a text wrapper, not the model, and that would not be a fair comparison, since we could tune the wrapper a bunch. So we did the closest thing we could instead: we maintained the frozen dataset and deterministic scorer but built a separate audited runtime that speaks Jev's native interface (two parallel Choice questions per case, plus a confidence gate that flips uncertain answers to refusal). The numbers below sit *next to* the chat results, not merged with them. Full evidence and the replay code are in the public repository.

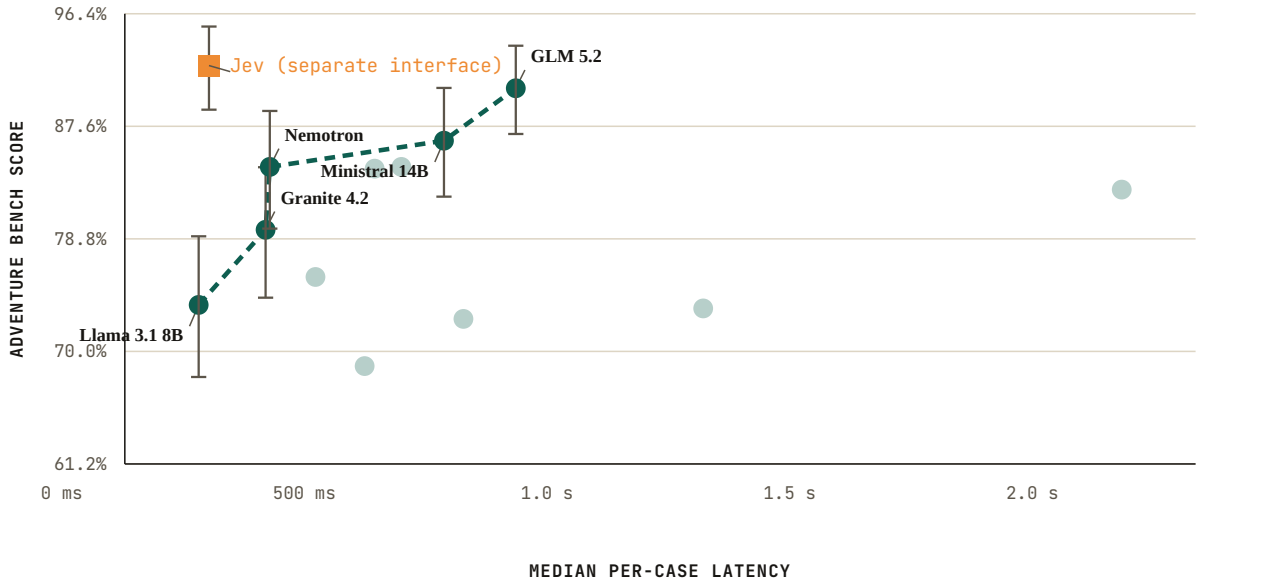
SCORE VERSUS COST



breaking the benchmark on speed as well

Jev's median per-case time is 303 ms. At 300 milliseconds, model judgment fits inside interactive loops: game turns, form validation, live moderation, anything where a human is waiting. At three seconds it introduces enough latency that the friction is felt quite significantly. This is client-measured round-trip time from one collection window, not a hardware benchmark, and the separate interface means it does not enter the chat-model frontier.

SCORE VERSUS LATENCY



a second pass after tuning on synthetic data

Across 244 distinct cases evaluated 3 times each (732 scored attempts), Jev achieves 0.923 accuracy (case-clustered 95% interval: 0.889 to 0.954) at a recorded cost of $0.028. A second, pinned `jev-1.13.0` run scores 0.925 (677/732).[1] At this benchmark's scale, where a full run costs pennies, small chat models are brutally efficient and Jev's per-token price advantage mostly washes out against its longer structured requests.

Every Jev answer carries a confidence estimate, and our runtime refuses to act below a threshold. This second pass is a new prompt specification, selected on synthetic data and frozen before the evaluation release: missing targets become unclear, mapping tags use a 0.75 gate, calibration tags use a 0.9 gate, and relative directions without orientation are refused. The gate a case gets is chosen from the case's pattern label, which the runtime sees but a deployed system reading only the player's command and the described scene would not, so these numbers measure the model together with that routing rather than the model alone. The historic v1 evidence still replays under its pinned v1 mapping; this is not a rewrite of the published result.

Importantly for the purposes of this benchmark, the release was not used to tune those rules. The failure mode is still instructive: 47 of 56 misses in the alias run are overcautious refusals rather than confident mistakes. A model whose characteristic failure is saying "I don't know" is better than one that is overconfident in its answers.

where it genuinely wins, and where it loses

The per-pattern table is the most Jev-shaped result in this piece:

pattern	attempts passed	attempts total
Out-of-vocabulary verbs	126	126
Absent objects	30	30
Exact verbs	36	36
Typos	35	36
Abbreviations	9	30
Missing prepositions	21	27

Refusal calibration is essentially perfect: unknown verbs, missing objects and nonsense all get refused at 100 percent, with zero type errors by construction. The weakness is terse commands. Single letters and dropped prepositions carry so little evidence that confidence sags below the gate, and good mappings get refused along with the bad ones. That is the price of the confidence tuned dial, and it suggests the next improvement is not a better model but state design: give the judgment more context and the same confidence goes further. When Jev does refuse, it is usually right (precision 0.862), and it catches nearly all genuine refusals (recall 0.980).

how this could be wrong

1. This is one model snapshot (pinned jev-1.13.0, also tested under its floating alias with 721 of 732 case/repetition agreements); vendors like TypeSafe can change backends and prices move.
2. Latency is client-measured round-trip time from one collection window, not a hardware benchmark.
3. The comparison set is twelve chat models on one 244-case benchmark. It says nothing about reasoning tasks, open-ended generation, or anything where strings are the actual product (this is the point though).

reproduce it!

The release pins every number above to raw evidence: the Jev v2 release record. Recheck it offline with `adventure-bench-rescore --run-id 20260917-jev-v2-full --check`. The runtime, frozen prompt specifications, and replay code live in the public benchmark repo, and TypeSafe documents the model at typesafe.ai with live docs at docs.typesafe.ai.

provenance

article output	source	verified
V2 score, interval, cost, and alias latency	`results/jev_decisions/figures.json`, generated from the audited v2 alias release and raw evidence	Yes

article output	source	verified
Pinned confirmation score and agreement	`results/jev_decisions/figures.json`, generated from the audited v2 pinned release and raw evidence	Yes
Per-tag counts and refusal metrics	`results/jev_decisions/figures.json`, generated from replay-derived v2 results	Yes
Cost and latency charts	Inline SVG generated by `scripts/jev_decisions/build_article.py` from `results/jev_decisions/figures.json`	Yes

1. The public alias run requested `jev-latest` and recorded `jev-1.13.0` in the provider response. The separate public confirmation run requested `jev-1.13.0` explicitly. A later provenance review found a differently hashed private collection labelled as the pinned run that instead requested `jev-latest`; it does not establish the source of the published confirmation. The public release remains the evidence used here, and its offline replay passes. The private-to-public lineage of the confirmation remains unresolved. ↩